Roma Tre

Università degli Studi

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche

Tesi di Laurea

Protecting the autonomic manager

A comparative analysis of Quorum and CometBFT as blockchain backends for Self-Protecting Systems

Corso di Laurea: Ingegneria Informatica

Candidato: Valerio Tatti

Matricola: 609824

Relatore: Prof. Stefano Iannucci

Anno Accademico: 2025/2026

Contents

1 Introduction **1**

1.1 The Paradigm of Self-Protecting Systems 1
1.2 The Vulnerability of the Autonomic Manager 2
1.3 Blockchain as a Mechanism of Distributed Trust 2
1.4 Objectives and Contributions of the Thesis 3
1.5 Structure of the Thesis 4

2 Technologies Setting and Related Works **5**

2.1 Autonomic Computing and the MAPE Cycle 5
2.1.1 The Four Aspects of Self-Management 6
2.1.2 The Structure of Autonomic Elements 7
2.1.3 Security Challenges 7
2.2 Blockchain in Permissioned Environments 8
2.2.1 Smart Contracts and Response Logic 8
2.2.2 Characteristics of Permissioned Environments 8
2.2.3 Alignment with SPS Requirements 9
2.3 Distributed Consensus Algorithms 10
2.4 State of the Art in Protecting the Manager 10

3 A Cyber-Resilient Architectural Model **11**

3.1 Architecture Overview 11
3.2 State Representation and Response Logic 11
3.3 Distributed Voting Mechanism 11
3.4 Threat Model 11

4 Requirement Alignment: Quorum vs. CometBFT **12**

4.1 Specific Requirements for SPS Applications 12
4.2 Analysis of GoQuorum 12
4.3 Analysis of CometBFT 12
4.4 Architectural Comparison 12

5 Implementation details **13**
5.1 Experimental Setup and Network Topology 13
5.2 Rust vs Solidity middleware 13
5.3 Alert detection and response loop 13
5.4 Blockchain config generation 13

6 Experiments and results **14**
6.1 Evaluation Methodology 14
6.2 Performance Analysis 14
6.2.1 Effectiveness of the Deduplication Mechanism 14
6.2.2 End-to-End Latency under Low Traffic 14
6.2.3 Transaction Throughput under Increasing Load 14

7 Conclusions and Future Work **15**
7.1 Summary of Research Findings 15
7.2 Limitations of the Study 15
7.3 Future Directions 15

List of Figures

List of Tables

Chapter 1

Introduction

In modern IT infrastructures, the rapid shift toward cloud-native, containerized microservices has dramatically expanded the digital attack surface. The escalation of sophisticated, high-velocity cyber threats and the ever-growing volume of sensitive data housed in these environments have driven the need for computing systems capable of defending themselves autonomously. Within this context, the autonomic computing paradigm has become a cornerstone of modern security engineering, aiming to reduce human operational burden and accelerate threat response. However, while automation resolves many latency and coordination challenges, it introduces new systemic vulnerabilities. If the automated security logic itself is targeted, it can become a vector for devastating privilege escalation and system-wide compromise.

1.1 The Paradigm of Self-Protecting Systems

Self-Protecting Systems [1] (SPS) are autonomic software systems designed to detect, analyze, and mitigate security threats dynamically with limited or no human intervention. Following the classic autonomic feedback loop, an SPS typically comprises two primary macro-components: a *managed system* and an *autonomic manager*. The managed system represents the production environment, services, or software applications exposed to potential attacks. Conversely, the autonomic manager serves as the central controller responsible for executing the security loop.

To achieve continuous protection, the autonomic manager organizes its operations into a pipeline governed by two main functional sub-components:

- **Intrusion Detection Systems (IDS):** Passive or active security sensors distributed across the managed system to monitor traffic, logs, or system calls and detect anomalous behavior or known attack signatures.
- **Intrusion Response Systems (IRS):** Decision-making engines responsible for selecting and executing the appropriate response actions (countermeasures)

to neutralize or mitigate the detected threats.

1.2 The Vulnerability of the Autonomic Manager

Although highly effective in mitigating threats directed at the managed system, this canonical design suffers from a critical structural vulnerability: the centralization of the autonomic manager. An adversary who penetrates the infrastructure can target the autonomic manager itself. By compromising this central controller, the attacker can silence intrusion alerts, disable response mechanisms, or even manipulate the manager into executing malicious countermeasures (e.g., disconnecting legitimate network segments or isolating critical database services).

Indeed, protecting the controller that protects the system remains an open and central challenge in autonomic security research. If the security logic is centralized and compromised, the integrity of the entire infrastructure is undermined, rendering the self-protecting features not only ineffective, but actively weaponized against the system.

1.3 Blockchain as a Mechanism of Distributed Trust

To eliminate the single point of failure represented by a centralized manager, our research [2] has proposed distributing the autonomic manager across a permissioned blockchain network. In this decentralized architecture, blockchain technology acts as a tamper-proof ledger and a distributed state machine (DSM) that logs security events and executes response logic securely through smart contracts.

This distributed approach establishes fundamental security guarantees that collectively safeguard the autonomic loop. To prevent a single compromised IDS sensor from injecting false alerts and triggering unauthorized countermeasures, the smart contract enforces a consensual voting mechanism. Under this scheme, updates to the shared security state are only committed once they receive corroborating reports from multiple independent, heterogeneous IDSs, ensuring that isolated sensor compromises do not lead to malicious actuations.

Also, by deploying the autonomic manager across a network of nodes running a Byzantine Fault-Tolerant (BFT) consensus protocol, the security loop gains strong structural resilience. This consensus mechanism allows the system to withstand the active compromise of up to 1/3 of the participating nodes encompassing both IDSs and validators while strictly maintaining both the availability and the integrity of the underlying ledger under partial network compromise.

Generally, the system is designed to reduce the attack surface as much as

possible treating actuators as black boxes (blind), the actuators receive signed mitigation commands directly from the blockchain state, rather than querying the state of potentially compromised application containers directly.

1.4 Objectives and Contributions of the Thesis

While the theoretical integration of blockchain into the SPS architecture provides outstanding security guarantees, the practical choice of the underlying blockchain technology is not neutral. A blockchain-based autonomic manager has unique requirements: it must handle sparse, highly event-driven alert traffic with exceptionally low latency and high determinism. Conversely, many features native to public blockchains (such as virtual-machine-level gas metering, token economics, transaction fees, and dynamic peer discovery) add unnecessary complexity, expand the trusted computing base (TCB), and increase the attack surface.

The natural baseline choice for permissioned enterprise setups is GoQuorum (a fork of Ethereum). However, this thesis argues that general-purpose smart contract platforms incur severe, non-intrinsic performance overheads due to interpreted virtual machine (EVM) bytecode execution, rigid transaction fee mechanisms, and fixed-interval block production. As an alternative, this work explores CometBFT (the successor of Tendermint), a lower-level BFT consensus engine that supports highly optimized, application-specific distributed state machines.

The primary objective of this thesis is to present a comprehensive qualitative and empirical comparison between a conventional general-purpose platform (GoQuorum) and a requirement-driven, application-specific alternative (CometBFT) to determine which architecture best supports the performance and cyber-resilience demands of modern Self-Protecting Systems.

Specifically, the core contributions of this thesis are as follows:

1. **Requirement Formulation:** We define and classify a comprehensive set of functional and non-functional requirements (required, desirable, and not needed) for blockchain technology in the specific context of an autonomic SPS loop.

2. **Architectural Analysis:** We conduct a thorough qualitative comparison of GoQuorum and CometBFT, highlighting the architectural mismatch of general-purpose virtual machines versus application-specific state machines.

3. **Prototype Implementation:** We design and implement a realistic, fully containerized SPS prototype emulated via Kathará on top of Docker. The prototype integrates three heterogeneous IDS engines (Snort, Suricata, and Zeek) protecting a vulnerable web application (OWASP Juice Shop).

4. **Deduplication Middleware:** We propose and develop a custom deduplication middleware layer for the member nodes. This layer filters repetitive alerts, preventing mempool saturation on the ledger keeping transaction number upper limited per block.

5. **Empirical Evaluation:** We perform extensive automated benchmarks under controlled attack conditions, comparing GoQuorum and CometBFT across key performance indicators, including end-to-end response latency, transaction throughput under heavy load, and CPU resource utilization.

1.5 Structure of the Thesis

To thoroughly explore the theoretical background, architectural design, and empirical evaluation of the proposed systems, the remainder of this thesis is structured as follows:

Chapter 2 – Technologies Setting and Related Work: Introduces the foundational concepts of autonomic computing, the MAPE cycle, Byzantine fault-tolerant consensus, the structural differences between public and permissioned blockchain systems, and reviews the state of the art in autonomic security.

Chapter 3 – A Cyber-Resilient Architectural Model: Details the technology-agnostic architecture of the Self-Protecting System, focusing on the interactions between IDSs, the smart contract state, and the blind actuators.

Chapter 4 – Requirement Alignment: Quorum vs. CometBFT: Formally identifies the blockchain properties strictly required for SPS applications and theoretically compares Quorum's general-purpose execution environment with CometBFT's application-specific design.

Chapter 5 – Implementation Details: Describes the prototype implementation, the containerized network topology emulated via Kathará, and the custom middleware developed for the evaluation.

Chapter 6 – Experiments and Results: Analyzes the performance of the two blockchain backends under varying workloads, presenting metrics on end-to-end latency, transaction throughput, and resource consumption.

Chapter 7 – Conclusions and Future Work: Summarizes the findings of the comparative analysis, acknowledges current limitations, and outlines directions for future research in decentralized system protection.

Chapter 2

Technologies Setting and Related Works

Before diving into the specific idea and implementation behind the system described in this research, it is necessary to introduce the basic theoretical concepts that inspired this architecture. These concepts will later provide us with the tools to comprehend by which metrics a blockchain-based autonomic agent should be measured, which paradigm first described autonomic systems, which security issues we can expect, how the current state-of-the-art (SOTA) solutions address them, and how our architecture compares to them.

2.1 Autonomic Computing and the MAPE Cycle

The paradigm of Autonomic Computing formally emerged in October 2001, when IBM released a manifesto identifying an upcoming software complexity crisis as the primary obstacle to further progress in the IT industry [3]. As computing environments began to weigh in at millions of lines of code, the expertise required for installation, configuration, and maintenance approached a limit on human capability to train new professionals. This challenge was exacerbated by the necessity of integrating several heterogeneous environments into corporate-wide systems and extending them globally across the Internet. IBM observed that relying solely on innovations in programming methods would not suffice to manage such massive and complex systems, which often require timely responses to rapid streams of changing demands.

To address this problem, IBM proposed the vision of autonomic computing: systems capable of managing themselves based on high-level objectives from human administrators [3]. The term "autonomic" was chosen for its biological connotation, tracing an analogy to the human autonomic nervous system. Just as the biological system governs vital, low-level functions such as heart rate and body temperature without conscious effort, autonomic computing aims to free human administrators from the burden of managing low-level system operations through

automation.

2.1.1 The Four Aspects of Self-Management

The essence of autonomic computing is self-management, intended to provide software that can adjust its behaviour in accordance with human objectives without their intervention [3]. IBM characterizes self-management through four fundamental aspects, often referred to as the "self-*" properties. Those principles have since materialized into concrete industry standards and architectural paradigms that govern contemporary network infrastructures:

- **Self-configuration**: Systems must automatically configure components and integrate themselves into complex environments following high-level policies. New components should be able to incorporate seamlessly, treating other components as black boxes. This modular perspective prefigured modern containerization paradigms, where container runtimes such as Docker have become the industry standard for application deployment.

- **Self-optimization**: Autonomic systems continually seek opportunities to improve their own performance and efficiency. In environments like complex middleware or databases that possess hundreds of manually set parameters, the system must monitor, experiment with, and tune these parameters dynamically. A notable industrial implementation of this concept is cloud-based auto-scaling (e.g., Amazon Web Services Auto Scaling), which dynamically adjusts resource provisioning to match fluctuating performance demands.

- **Self-healing**: The system is designed to automatically detect, diagnose, and repair localized software and hardware problems. Using knowledge of the system configuration and logs, the autonomic manager can identify root causes of failures, match them against known patches, and retest the system without manual debugging. In contemporary orchestration systems, self-healing is commonly implemented via automated health checks and liveness probes (e.g., in Kubernetes), which automatically restart or replace failing service instances.

- **Self-protection**: Autonomic systems defend the infrastructure as a whole against malicious attacks [4]. They use sensor reports to anticipate problems and take proactive steps to avoid or mitigate systemwide failures. An example of such proactive defense is represented by modern Web Application Firewalls (WAFs), such as Cloudflare's edge security systems, which autonomously detect, analyze, and block malicious traffic patterns.

2.1.2 The Structure of Autonomic Elements

Architecturally, autonomic systems are composed of elements that manage their own internal behavior and relationships with others in accordance with established policies [3]. An autonomic element typically consists of one or more managed elements coupled with a single autonomic manager.

- **Managed Element**: This component can be found in non-autonomic systems and can be a hardware resource (such as storage or a CPU) or a software resource (such as a database or a web service).
- **Autonomic Manager**: The manager distinguishes the autonomic element by monitoring the managed element and its environment, and executing actions based on the analysis of their information.

The internal logic of the autonomic manager is structured around a fundamental control loop known as the **MAPE-K** cycle [3, 5]:

1. **Monitor**: Collects, aggregates, and filters information from the managed element and its environment.
2. **Analyze**: Matches a diagnosis on the system's high-level status based on monitored information and triggers a response.
3. **Plan**: Creates or selects a sequence of actions that matches the organization's objectives.
4. **Execute**: Applies the derived plan / action to the managed element through effectors.
5. **Knowledge**: A central repository for policies, historical data, and system configurations that supports the other four phases defining the organization's objectives.

2.1.3 Security Challenges

Autonomic elements, unlike typical Web services, do not honor requests unconditionally; they provide services only if doing so is consistent with their internal goals [3].

This high degree of autonomy introduces critical system-level issues that cannot be ignored when evaluating attack surfaces. Because these systems rely on high-level goals, they are vulnerable to new forms of attack where an adversary might manipulate policies to gain systemic leverage [6]. This highlights the necessity for a secure-by-design management infrastructure that can address the "Who watches the watchers?" dilemma of autonomic elements [6, 7].

2.2 Blockchain in Permissioned Environments

The integration of blockchain technology to register system state represents an innovative approach to ensuring cyber-resilience [2]. However, in this specific application context, the choice of the underlying blockchain platform is not neutral, and it is necessary to define which blockchain properties are beneficial for a Self-Protecting System. A fundamental distinction must be made between public (permissionless) blockchains and private (permissioned) blockchains [8].

2.2.1 Smart Contracts and Response Logic

Before analyzing network access models, it is essential to define the mechanism through which the blockchain enforces security policies: the *smart contract*. A smart contract is a self-executing distributed software deployed on a blockchain that automatically enforces predefined rules and agreements between nodes without the need for intermediaries [9].

Once deployed, smart contracts run deterministically across all nodes in the network, ensuring the transparency, trustlessness, and immutability of the encoded logic [9]. In the context of a Self-Protecting System (SPS), the integrity of the smart contract's state and its execution is guaranteed by the inherent properties of the blockchain itself [2]. This enables the autonomic manager to maintain a consistent, system-wide view and coordinate responses even under partial compromise [2].

2.2.2 Characteristics of Permissioned Environments

Public blockchains are designed as open networks where anyone can participate. In contrast, a permissioned environment requires nodes to authenticate each other, often within a fixed trust boundary [8]. This architectural difference fundamentally alters the threat model and the system's performance requirements.

In a permissioned network, many attacks traditionally critical for permissionless blockchains, such as Sybil and Eclipse attacks, are naturally mitigated because participant identities are controlled and membership is typically static [2].

Since permissionless environments adopt a fundamentally "trustless" model, they must employ strict, and often resource-expensive, consensus algorithms. In a permissioned blockchain, consensus can be simplified because all participating nodes are pre-vetted, identified, and operate within a predefined deployment. Consequently, the economic dynamics, gas fees, token incentives, and game-theory mechanisms typical of public cryptocurrencies designed to coordinate anonymous actors become functionally obsolete [10]. Furthermore, general-purpose virtual machines (VMs) such as the Ethereum Virtual Machine (EVM) often introduce unnecessary complexity, processing overhead, and a broader attack

surface when the system only requires a constrained, deterministic runtime for state transitions [10].

Instead, the focus in such environments shifts towards a specific set of essential properties. As highlighted in recent literature evaluating blockchain implementations for Self-Protecting Systems [10], the required features are:

- **Byzantine Fault-Tolerant (BFT) consensus with strong finality**: To tolerate a bounded number of compromised nodes and commit updates deterministically.
- **Static permissioned membership**: To match the fixed trust boundary of the deployment.
- **Cryptographic transaction signing**: To ensure the authenticity and accountability of alerts submitted by IDSs.
- **Event-driven block production**: To optimize resource usage by generating blocks only when pending transactions (e.g., security alerts) are present, rather than continuously producing empty blocks during idle periods.

Additionally, desirable properties include in-memory replicated state handling (as the shared state in an SPS is typically small), low and predictable latency to guarantee timely countermeasures, and push-based notifications to immediately trigger actuators upon state updates [10].

2.2.3 Alignment with SPS Requirements

For an SPS, the blockchain's sole functions are:

- To gather a shared state from Intrusion Detection Systems (IDS) [2].
- To ensure the reliable functioning of the response logic even in the presence of security faults in the managed system or in the autonomic manager [2].
- To communicate the response logic's decisions to the actuators securely [2].

Under these assumptions, permissioned environments offer indispensable advantages, such as static membership that matches the trust boundaries of a real-world deployment [10]. Furthermore, cryptographic transaction signing ensures the authenticity and accountability of sensor submissions [2, 10]. Finally, while blockchain provides an immutable audit trail for forensics, permissioned settings may allow for configurable data-retention policies, which is particularly relevant for resource-constrained or embedded systems [10].

Ultimately, the relevant blockchain properties in this domain are not openness or tokenization, but minimality, determinism, and resilience [10].

2.3 Distributed Consensus Algorithms

2.4 State of the Art in Protecting the Manager

Chapter 3

A Cyber-Resilient Architectural Model

3.1 Architecture Overview

3.2 State Representation and Response Logic

3.3 Distributed Voting Mechanism

3.4 Threat Model

Chapter 4

Requirement Alignment: Quorum vs. CometBFT

4.1 Specific Requirements for SPS Applications

4.2 Analysis of GoQuorum

4.3 Analysis of CometBFT

4.4 Architectural Comparison

Chapter 5

Implementation details

5.1 Experimental Setup and Network Topology

5.2 Rust vs Solidity middleware

5.3 Alert detection and response loop

5.4 Blockchain config generation

Chapter 6

Experiments and results

6.1 Evaluation Methodology

6.2 Performance Analysis

6.2.1 Effectiveness of the Deduplication Mechanism

6.2.2 End-to-End Latency under Low Traffic

6.2.3 Transaction Throughput under Increasing Load

Chapter 7

Conclusions and Future Work

7.1 Summary of Research Findings

7.2 Limitations of the Study

7.3 Future Directions

Bibliography

[1] Eric Yuan, Naeem Esfahani, and Sam Malek. "A Systematic Survey of Self-Protecting Software Systems". In: *ACM Trans. Auton. Adapt. Syst.* 8.4 (Jan. 2014). ISSN: 1556-4665. DOI: 10.1145/2555611. URL: https://doi.org/10.1145/2555611.

[2] Tommaso Caiazzi et al. "A Novel Architecture for Cyber-Resilient Self-Protecting Systems Based on Blockchain". In: *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. 2025, pp. 1–8. DOI: 10.1109/COMPSAC65507.2025.00010.

[3] Jeffrey O Kephart and David M Chess. "The vision of autonomic computing". In: *Computer* 36.1 (2003), pp. 41–50.

[4] Eric Yuan, Naeem Esfahani, and Sam Malek. "A systematic survey of self-protecting software systems". In: *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 8.4 (2014), pp. 1–41.

[5] Paolo Arcaini, Elvinia Riccobene, and Patrizia Scandurra. "Modeling and Analyzing MAPE-K Feedback Loops for Self-Adaptation". In: *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 2015, pp. 13–23. DOI: 10.1109/SEAMS.2015.10.

[6] David M Chess, Charles C Palmer, and Steve R White. "Security in an autonomic computing environment". In: *IBM Systems journal* 42.1 (2003), pp. 107–118.

[7] Irdin Pekaric et al. "A systematic review on security and safety of self-adaptive systems". In: *Journal of Systems and Software* 203 (2023), p. 111716.

[8] Xin Wang et al. "Bft in blockchains: From protocols to use cases". In: *ACM Computing Surveys (CSUR)* 54.10s (2022), pp. 1–37.

[9] Vitalik Buterin et al. "Ethereum white paper". In: *GitHub repository* 1 (2013), pp. 22–23.

[10] Simone Albero et al. "Which Blockchain Technology Best Supports Self-Protecting Systems? A comparison between Quorum and CometBFT". In: *Proceedings of the 8th Distributed Ledger Technology Workshop (DLT 2026)*. In press. Pula, Italy: CEUR-WS.org, June 2026.